Create Kubernetes Manifests

Mohammed Dawoud Mota

```
[ec2-user@ip-172-31-42-120 manifests]$ cat << EOF > flask-service.yaml
---
apiVersion: v1
kind: Service
metadata:
  name: nextwork-flask-backend
spec:
  selector:
    app: nextwork-flask-backend
  type: NodePort
  ports:
    - port: 8080
      targetPort: 8080
      protocol: TCP
EOF
```

Introducing Today's Project!

Today I am here to create the Kubernetes manifest files that will define exactly how my backend application should be deployed and managed inside my EKS cluster. This includes setting up both the Deployment manifest, which instructs Kubernetes on how to run and maintain multiple replicas of my containerized backend, and the Service manifest, which determines how the application will be exposed so users and other services can reach it. By completing these steps, I ensure that Kubernetes has clear, consistent instructions for deploying my backend reliably across the cluster.

Tools and concepts

The key tools and concepts in this project were AWS services like EC2 for building the environment, ECR for storing the container image, and EKS for running the Kubernetes cluster; containerization tools like Docker and the project’s Dockerfile to package the backend; Kubernetes components such as Deployments, Services, and manifests to define and expose the application; and supporting tools like Git, eksctl, and IAM roles that enabled cluster creation, code retrieval, and secure access to AWS resources.

Project reflection

I chose to do this project today to learn more about EKS and Kubernetes, specifically learning more about Kubernetes manifests.

This project took me approximately 45 minutes to complete.

Project Set Up

Kubernetes cluster

To set up my Kubernetes cluster, I launched a new EC2 instance in my AWS account and connected to it using EC2 Instance Connect so I could run commands directly from the terminal. I installed eksctl, which is the command line tool that simplifies creating and managing EKS clusters, and verified the installation. Since the EC2 instance initially has no permissions, I created an IAM role with AdministratorAccess and attached it to the instance so it could interact with AWS services. With the environment prepared and the correct permissions in place, I used eksctl from the EC2 terminal to create the EKS cluster that will host my backend application.

Backend code

I retrieved the backend code by installing Git on my EC2 instance, configuring it with my name and email, and then cloning the team member's GitHub repository directly into the instance. After opening the repository page in a browser, I copied the HTTPS clone URL and used the git clone command in the terminal to pull down the full nextwork-flask-backend project. Once the clone completed, I verified the files by navigating into the new directory and listing its contents, which included the app.py file, the Dockerfile, the requirements.txt file, and the README. This gave me a complete and ready-to-use copy of the backend code that I would later containerize and deploy.

Container image

I built the container image by first installing Docker on my EC2 instance, starting the Docker service, and adding the ec2-user to the Docker group so I could run Docker commands smoothly. Once Docker was ready, I navigated into the backend directory that I cloned from GitHub and used the Dockerfile provided in the project to build the image. By running the docker build command and pointing it to the current directory, Docker read the Dockerfile and packaged the

This image became the standardized unit that Kubernetes will later pull and use to run identical containers across the cluster.

I pushed my container image by first creating a new Amazon ECR repository and then authenticating Docker to ECR using the login command provided in the console. After confirming the repository was ready, I tagged my locally built image with the latest label so Kubernetes could easily reference the correct version. Once the image was tagged, I used the Docker push command to upload it into the ECR repository. After refreshing the ECR console, I verified that the image was successfully stored, making it available for Kubernetes to pull during deployment.

Manifest files

A Kubernetes manifest is a configuration file that describes the desired state of an application inside a Kubernetes cluster. It tells Kubernetes exactly how the application should run, including details such as which container image to use, how many replicas to maintain, what labels to apply, and how the application should be exposed or connected to other components. By defining these instructions in a manifest, Kubernetes can automatically create, update, and manage the resources needed to keep the application running reliably and consistently across the cluster.

A Kubernetes deployment defines the desired state of my application and ensures that Kubernetes continuously maintains that state across the cluster. It specifies how many replicas of my backend should run, which container image to use, and the labels that identify the pods created by the deployment. Once applied, Kubernetes uses this manifest to launch and manage the pods, replace any that fail, and keep the application running reliably and consistently.


GNU nano 8.3
flask-deployment.yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
name: nextwork-flask-backend
namespace: default
spec:
replicas: 3
selector:
matchLabels:
app: nextwork-flask-backend
template:
metadata:
labels:
app: nextwork-flask-backend
spec:
containers:
- name: nextwork-flask-backend
image: 289654514139.dkr.ecr.us-east-1.amazonaws.com/nextwork-flask-backend:latest
ports:
- containerPort: 8080
[Read 21 lines]
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^B Where Was

Service Manifest

A Kubernetes Service acts as the stable access point that routes traffic to the correct pods running inside the cluster. Since pods can change over time as Kubernetes replaces or scales them, the Service provides a consistent way for users or other systems to reach the application without needing to know the internal pod details. It selects the appropriate pods using labels and exposes the backend on a specific port so external traffic can reliably reach the application.

My Service manifest defines how Kubernetes should expose my backend application and route traffic to the correct pods. It specifies a Service named nextwork-flask-backend that selects any pod labeled with app: nextwork-flask-backend, ensuring that traffic is consistently directed to the right workload even if pods are replaced or scaled. The Service is configured as a NodePort, which allows external users to reach the application through a port on any worker node in the cluster. It listens on port 8080 and forwards that traffic to the same target port inside the pods, creating a stable and reliable entry point for accessing the backend.

```
[ec2-user@ip-172-31-42-120 manifests]$ cat << EOF > flask-service.yaml
---
apiVersion: v1
kind: Service
metadata:
  name: nextwork-flask-backend
spec:
  selector:
    app: nextwork-flask-backend
  type: NodePort
  ports:
    - port: 8080
      targetPort: 8080
      protocol: TCP
EOF
```

The place to learn & showcase your skills

Check out nextwork.ai for more projects